IDENTIQ – FACE RECOGNITION

A project report Submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE ENGINEERING

By

Munnangi Janvika

2200031805

Under the esteemed guidance of

Mr. Sharath Babu

HR & Brand Promotion,

GAINT Clout Technologies Pvt Ltd

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

K L UNIVERSITY

Green Fields, Vaddeswaram, Tadepalli,

Guntur - 522 502, Andhra Pradesh.

2025-26

GAINT Clout Technologies Pvt Ltd

Awfis Elite 4th floor, Orbit Building, Knowledge City,

Hyderabad

Bonafide Certificate

*This is to certify that this Interim report entitled **"IDENTIQ – Face Recognition"** submitted to **Department of Computer Science Engineering, KL Deeemed to be University, Guntur,** in connection with the University internship program is a bonafide record of work done by "**MUNNANGI . JANVIKA"** under my supervision at the **"GAINT Clout Technologies Pvt Ltd**".*

Sharath Babu

HR & Brand Promotion

K L Deemed to be UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Certificate

This is Certified that the project entitled "IDENTIQ - Face Recognition Project" which is a theoretical work carried out by Munnangi Janvika(2200031805),in partial fulfillment for the award of the degree of Bachelor of Technology in Department ***Computer Science Engineering****, during the year* ***2025-26****. The project has been approved as it satisfies the academic requirements.*

Department Coordinator ***Head of the Department***

University Guide **External**

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to **Mr.K.Prasad, Chairman,** and **K.Srinivasa Rao ,CEO of GAINT Clout Technologies Pvt Ltd** for providing me with the valuable opportunity to undergo my internship and gain practical exposure to real-world industry practices. This experience has been immensely beneficial in enhancing my technical skills and professional knowledge.

I extend my sincere thanks to my company mentors — **Mr.Sharath Babu**, **Ms. Rohini, Mr.Elisha, Ms.Nikitha** for their constant support, guidance, and encouragement throughout the course of my internship. Their valuable insights and feedback have played a crucial role in the successful completion of my project.

I am deeply grateful to my **University Guide, Dr.Ch.Venkateswarao**, for his continuous support during the entire internship period. Our special thanks to **Dr. P. V. Chalapathy**, Director of Practice School, and **Dr. B. Suresh**, Practice School In-Charge, for providing us with this invaluable learning experience. We are also grateful to **Mrs. K. Swathi**, Department Coordinator, for her continuous support and coordination.

Our project, titled **"Identiq – Face Recognition Project"** was successfully completed as part of our Practice School program at **Gaint Clout Technology Pvt. Ltd.** The team comprised **V. Basaveswara Chowdary** as the **Team Lead and Front End**, **M. Janvika** and **K. Sri Varsha** as the **Back-End Developers**.

We sincerely appreciate the collaborative efforts, technical mentorship, and teamwork that contributed to the successful completion of this project.

Project Associates

2200080133 - V.BasaveswaraChowdary

2200031805 – M.Janvika

2200080198 – K.Sri Varsha

ABSTRACT

Face recognition is a rapidly growing area of biometric identification that leverages computer vision and machine learning techniques to automatically detect and identify human faces. This project presents the design and implementation of a face recognition system capable of accurately recognizing and verifying individuals from digital images or live video streams. The system employs techniques such as face detection and face embedding extraction using deep learning models like FaceNet or OpenFace. The extracted features are then compared using distance metrics to determine identity matches. The project demonstrates the potential of face recognition technology in applications such as security systems, attendance monitoring, and access control. Experimental results indicate that the system achieves high accuracy under varying lighting conditions and facial expressions, proving its effectiveness and reliability in real-world scenarios.

The backend of the face recognition project was developed using Python as the core programming language due to its strong support for machine learning and image processing libraries. Flask was used to create a lightweight web framework for handling requests between the user interface and the recognition model. The face detection and recognition processes utilized libraries such as OpenCV, NumPy, and face_recognition for efficient image handling and feature extraction.

TABLE OF CONTENTS

S. NO	TITLE	PAGE NO
1	Certificate	3
2	Acknowledgement	4
3	Abstract	5
4	Table of contents	6
5	List Of Symbols	
6	Introduction	
7	Literature Review	
8	Modules and Features	
9	Implementation and File structure	
10	List of Figures	
11	Discussion	
12	Conclusion	

List of Symbols, Abbreviations

Symbol / Abbreviation	Description
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
DB	Database
API	Application Programming Interface
GUI	Graphical User Interface
Encodings	Numerical representations of facial features used for recognition
OpenCV	Open Source Computer Vision Library
JSON	JavaScript Object Notation (data format used in APIs)

CHAPTER 1
INTRODUCTION

Face recognition is one of the most widely used and rapidly evolving applications of computer vision and artificial intelligence. It enables computers to automatically identify or verify a person from a digital image or a video frame, based on the unique features of their face. With the growing demand for secure and efficient identification systems, face recognition technology has become an essential tool in areas such as security monitoring, access control, attendance tracking, and user authentication. Unlike traditional methods that rely on passwords or ID cards, face recognition offers a contactless, reliable, and user-friendly solution for identity verification.

This project focuses on developing a face recognition system that can accurately detect and recognize human faces using image processing and machine learning techniques. The system utilizes Python along with libraries such as OpenCV and face_recognition to perform tasks like face detection, feature extraction, and matching. A Flask-based backend manages user data and recognition results, while a simple graphical interface allows users to interact with the system easily. The main objective of this project is to design a cost-effective, efficient, and scalable recognition system that can be integrated into real-world applications such as smart attendance systems and security access controls.

Problem Statement

In today's rapidly advancing digital world, the increasing reliance on technology has created a demand for efficient, secure, and intelligent systems that can address real-world challenges. However, many existing solutions fail to effectively meet user needs due to issues such as poor scalability, lack of automation, inefficient data handling, and limited accessibility. This project aims to identify and address these shortcomings by developing an innovative, reliable, and user-centric solution that leverages modern technologies to enhance performance, usability, and overall system efficiency.

Major Problems Identified (Backend)

1. Complex Integration Between Modules

Integrating multiple backend services such as authentication, database management, and API communication was challenging. Ensuring smooth data flow between modules without breaking existing functionality required careful design and debugging.

2. Inefficient Database Structure and Query Handling

In the early stages, improper database schema design caused redundant data storage and slower query responses. Optimizing relationships between tables and implementing indexing was necessary to improve performance and reduce server load.

3. Authentication and Security Issues

Implementing a secure login and face recognition authentication system involved challenges in managing encrypted data, handling JWT tokens, and preventing unauthorized access. Ensuring that sensitive user data was securely stored and transmitted was a major concern.

4. API Response Delays and Server Performance

The backend experienced response delays when handling multiple requests simultaneously. Issues such as unoptimized API endpoints, lack of caching, and limited server capacity affected overall system performance and user experience.

5. Error Handling and Exception Management

Initially, the backend lacked a proper error-handling mechanism, causing the system to crash or return unclear messages when an issue occurred. Implementing structured exception handling and meaningful response codes became essential for stability and easier debugging.

6. Data Synchronization and Consistency

Maintaining consistent data between different services (e.g., user info, orders, and logs) was difficult during concurrent operations. Race conditions and unsynchronized updates occasionally led to mismatched or outdated records.

Objectives

1. **To Develop a Secure and Efficient Backend System**

Design and implement a robust backend architecture capable of handling user authentication, data storage, and service requests with high reliability and security.

2. **To Enable Smooth Integration Between Multiple Services**

Build modular APIs that support seamless communication between different application modules, such as user management, order processing, and payment systems.

3. **To Optimize Database Performance and Scalability**

Create an efficient and normalized database structure to ensure fast query execution, minimal redundancy, and easy scalability as the number of users and services grow.

4**. To Implement Secure Authentication Mechanisms**

Integrate advanced authentication techniques such as token-based or biometric verification to enhance system security and prevent unauthorized access.

5**. To Ensure Real-Time Data Processing and Notifications**

Enable real-time updates for users and admins by integrating push notifications, live order tracking, and event-driven backend workflows.

6. **To Improve System Reliability Through Error Handling**

Develop a robust error-handling framework that ensures consistent server performance, meaningful error messages, and smooth recovery from failures.

7. **To Provide Analytics and Insights for Business Users**

Implement backend logic for data aggregation and analytics to support dashboards that display key performance indicators like orders, revenue, and user activity.

Scope and Limitations

Scope:

1. Secure and Scalable Backend Architecture

• Developed using frameworks like Node.js/Django/FastAPI to handle authentication, data storage, and service management.

• Ensures scalability to support multiple concurrent users and transactions efficiently.

2. API Development and Integration

• RESTful APIs designed for communication between frontend, database, and external services.

• Provides endpoints for authentication, order management, payments, and notifications.

3. Database Management

• Structured database schema (MySQL/MongoDB) for storing users, products, and orders efficiently.

• Ensures fast queries, minimal redundancy, and secure storage of sensitive data.

4. Authentication and Authorization

• Implementation of JWT or OAuth-based authentication for secure user access.

• Role-based access control for admins, vendors, and customers.

5. Real-Time Processing and Notifications

• Supports real-time order tracking and status updates using WebSockets or Firebase.

• Integrates with push notification services to alert users about order progress and system updates.

Limitations:

1. Server and Hosting Dependencies

• System performance depends on server specifications and hosting quality.

• Limited server resources may cause slow responses during high traffic.

2. Real-Time Communication Constraints

• Real-time features like live tracking rely on stable internet and WebSocket connections.

• Any network disruption can delay updates or break live sessions.

3. Limited Fault Tolerance

• If database or API servers go down, core functionalities like authentication or order updates may fail.

• Requires proper backup and load-balancing setup for high availability.

4. Third-Party API Reliance

• Payment, notification, and analytics modules depend on external APIs (e.g., Razorpay, Firebase).

• Any downtime or policy change in these services can affect backend operations.

5. Data Privacy and Compliance Challenges

• Although encryption is implemented, complete compliance with international data privacy laws (like GDPR) may not be covered.

6. Scalability and Maintenance

• Scaling to millions of users requires advanced cloud setup and database sharding.

• Regular maintenance is needed to handle version updates, API changes, and bug fixes.

CHAPTER 2

LITERATURE REVIEW

The backend plays a crucial role in ensuring the stability, scalability, and security of on-demand delivery applications. It manages data flow, real-time communication, user authentication, and third-party integrations such as payment gateways and notifications. Modern backend systems are designed to handle large volumes of concurrent users while maintaining low latency and high reliability.

1. Early face recognition systems used basic image comparison methods such as Eigenfaces and Fisherfaces, which relied on linear algebra techniques to detect similarities between facial images. However, these traditional methods were sensitive to lighting, facial angles, and background variations, which reduced accuracy.

   With the rise of deep learning and convolutional neural networks (CNNs), modern systems now achieve high accuracy by automatically learning facial features from large datasets.
2. Role of Artificial Intelligence and Deep Learning
3. Recent research focuses on using AI-based models like VGG-Face, FaceNet, and OpenFace to extract deep facial embeddings. These models can recognize faces even when there are small changes in expression, lighting, or pose. Studies show that CNN-based models outperform older techniques by a large margin in both speed and accuracy.

   Many modern authentication systems now use real-time face detection through frameworks like OpenCV, combined with deep learning models for recognition and verification.

4. Face Detection Techniques
5. Before recognizing a face, the system must first detect it. Algorithms like Haar Cascade Classifier and Multi-Task Cascaded Convolutional Networks (MTCNN) are commonly used. Haar Cascades are fast and suitable for simple applications, while MTCNN provides more accurate detection by handling complex lighting and multi-face scenarios.
6. 4. Security and Authentication
7. Face recognition adds an extra layer of security compared to traditional password-based systems. Unlike passwords, faces cannot be easily stolen or forgotten. Studies have shown that biometric authentication, especially facial recognition, reduces risks of unauthorized access and identity theft.

   However, researchers also highlight the need for liveness detection to prevent misuse through photos or videos of a user.
8. 5. Real-Time Processing and Performance
9. Real-time recognition is an important aspect of practical systems. With improvements in computing power and optimized libraries, real-time face detection can now be achieved using devices with basic hardware. Integration with cloud services or backend servers further enhances speed and scalability.
10. 6. Applications and Future Trends
11. Face recognition is now used in various domains like secure login systems, smart attendance tracking, border security, and surveillance. New trends include combining face recognition with emotion detection or voice verification to build multi-modal authentication systems that are even more secure and intelligent.

CHAPTER 3

MODULES AND FEATURES

1. **User Registration Module**
   - Allows new users to sign up by entering details like name, email, and password.
   - Captures and stores multiple facial images of each user for training and recognition.

2. **Face Detection Module**
   - Detects human faces from live camera input or uploaded images.
   - Uses algorithms like Haar Cascade or MTCNN for accurate detection in different lighting conditions.

3. **Face Recognition Module**
   - Compares captured faces with stored data to verify identity.
   - Uses deep learning models (like CNN or FaceNet) to extract and match facial features.

4. **Authentication Module**
   - Grants or denies access based on recognition results.
   - Ensures only authorized users can log in or access the system securely.

5. **Database Management Module**
   - Stores user information, facial data, and login records securely.
   - Provides quick retrieval and efficient management of data for faster processing.

6. **Admin Module**
   - Allows admin to view registered users, manage data, and monitor system activity.
   - Can update, delete, or verify user records if needed..

7. **Camera Interface Module**
   - Connects the system to the webcam or external camera for real-time image capture.
   - Ensures clear image quality for better recognition accuracy.

CHAPTER 4

IMPLEMENTATION AND FILE STRUCTURE

Implementation Overview

Backend setup:

1. Create a new project folder and set up a Python virtual environment.
2. Install required libraries: OpenCV, face-recognition, dlib, numpy, face_recognition.

   Organize folders for each module: Batch Encoding, Face Utils, Real-Time Recognition, Register User, and Update User.

Installation steps:

- Install Python 3.8 or above on your system.
- Create and activate a virtual environment for the project.
- Install all dependencies
- Register a new user through the camera and verify recognition in real time.
- Check logs or console output to confirm successful authentication.

Work Flow

Start Application→ User Registration→ Batch Encoding Module→ Real-Time Recognition→ Authentication→ Update User Module→ Error Handling and Logging→ End of Process

File Structure

```
FACE_RECOGNITION_PROJECT/
│
└── face_recognition_project/
    ├── __pycache__/
    ├── .vscode/
    ├── data/
    ├── faces/
    ├── faces_expressions/
    ├── faces_organized/
    ├── logs/
    ├── models/
    ├── outputs/
    ├── sample_data/
    ├── test_images/
    ├── test_single_folder/
```

```
│
├── batch_encoder.py
├── emotion_demo.py
├── encoding_test.py
├── face_manager.py
├── face_utils.py
├── gesture_module.py
├── integration.py
├── main.py
├── PROJECT_SUMMARY.md
├── realtime_recognition.py
├── regen_encodings.py
├── register_user.py
└── update_user.py
```

Key Files and Directories

Files

File Name	Purpose
main.py	The central file that runs the overall project; connects modules and initiates recognition.
realtime_recognition.py	Handles real-time face recognition using webcam or live video feed.
register_user.py	Registers new users by capturing and encoding their facial data.
update_user.py	Updates or modifies existing user face data or re-encodes new images.
batch_encoder.py	Encodes multiple faces in batch mode and stores them in the database.
face_utils.py	Contains utility functions for face detection, encoding, comparison, and verification.
face_manager.py	Manages user face data (loading, saving, matching, etc.).
integration.py	Integrates all modules (face, gesture, and emotion recognition).

Directories

- face_recognition_project/data/ → Contains datasets and sample data used for training and testing face recognition models.
- face_recognition_project/faces/ → Stores images of registered users captured during the registration process.
- face_recognition_project/faces_expressions/ → Contains data or models for detecting facial expressions (emotion recognition).
- face_recognition_project/faces_organized/ → Maintains organized folders for each user's facial data.
- face_recognition_project/logs/ → Stores log files that record recognition results, errors, and system activities.
- face_recognition_project/models/ → Holds pre-trained models used for face detection, encoding, or emotion recognition.
- face_recognition_project/outputs/ → Contains output images, reports, or recognition results generated by the system.

1. batch_encoder

```python
#!/usr/bin/env python3
"""
Batch Face Encoding Utility
Encodes all face images in a folder and generates a .pkl file with encodings.
"""

import os
import pickle
import cv2
import face_recognition
import numpy as np
from datetime import datetime

def encode_folder_to_pkl(input_folder, output_pkl="data/encodings.pkl", validate_images=True):
    """
    Encode all face images in a folder and save to .pkl file.

    Args:
        input_folder (str): Path to folder containing face images
        output_pkl (str): Path to output .pkl file
        validate_images (bool): Whether to validate images before encoding

    Returns:
        dict: Summary of encoding results
    """
    print(f"[INFO] Starting batch encoding from: {input_folder}")
    print(f"[INFO] Output file: {output_pkl}")

    # Create output directory if it doesn't exist
    os.makedirs(os.path.dirname(output_pkl), exist_ok=True)

    # Supported image formats
    supported_formats = ('.jpg', '.jpeg', '.png', '.bmp', '.tiff')

    # Results tracking
    results = {
        'total_images': 0,
        'successful_encodings': 0,
        'failed_encodings': 0,
        'users_encoded': set(),
        'errors': []
    }

```

```python
    # Dictionary to store encodings
    encodings_dict = {}

    # Process each user folder
    for user_folder in os.listdir(input_folder):
        user_path = os.path.join(input_folder, user_folder)

        if not os.path.isdir(user_path):
            continue

        print(f"\n[INFO] Processing user: {user_folder}")
        user_encodings = []

        # Process each image in user folder
        for img_file in os.listdir(user_path):
            if not img_file.lower().endswith(supported_formats):
                continue

            img_path = os.path.join(user_path, img_file)
            results['total_images'] += 1

            print(f"  Processing: {img_file}")

            try:
                # Validate image if requested
                if validate_images:
                    if not validate_single_image(img_path):
                        results['failed_encodings'] += 1
                        results['errors'].append(f"Validation failed: {img_path}")
                        continue

                # Encode the face
                encoding = encode_single_face(img_path)

                if encoding is not None:
                    user_encodings.append({
                        'image_path': img_path,
                        'encoding': encoding,
                        'timestamp': datetime.now().isoformat()
                    })
                    results['successful_encodings'] += 1
                    print(f"    [OK] Encoded successfully")
                else:
                    results['failed_encodings'] += 1
                    results['errors'].append(f"No face found: {img_path}")
                    print(f"    [ERROR] No face detected")
```

```python

            except Exception as e:
                results['failed_encodings'] += 1
                results['errors'].append(f"Error processing {img_path}: {str(e)}")
                print(f"    [ERROR] Error: {str(e)}")

        # Store user encodings if any were successful
        if user_encodings:
            encodings_dict[user_folder] = user_encodings
            results['users_encoded'].add(user_folder)
            print(f"  [OK] User {user_folder}: {len(user_encodings)} encodings")
        else:
            print(f"  [ERROR] User {user_folder}: No successful encodings")

    # Save encodings to .pkl file
    if encodings_dict:
        try:
            with open(output_pkl, 'wb') as f:
                pickle.dump(encodings_dict, f)
            print(f"\n[SUCCESS] Encodings saved to: {output_pkl}")
        except Exception as e:
            print(f"[ERROR] Failed to save encodings: {str(e)}")
            return results
    else:
        print(f"\n[WARNING] No encodings to save")
        return results

    # Print summary
    print(f"\n=== ENCODING SUMMARY ===")
    print(f"Total images processed: {results['total_images']}")
    print(f"Successful encodings: {results['successful_encodings']}")
    print(f"Failed encodings: {results['failed_encodings']}")
    print(f"Users encoded: {len(results['users_encoded'])}")
    print(f"Success rate: {(results['successful_encodings']/results['total_images']*100):.1f}%" if results['total_images'] > 0 else "0%")

    if results['errors']:
        print(f"\nErrors encountered:")
        for error in results['errors'][:5]:  # Show first 5 errors
            print(f"  - {error}")
        if len(results['errors']) > 5:
```

```python
            print(f"  ... and {len(results['errors']) - 5} more errors")

    return results

def validate_single_image(image_path):
    """
    Validate that an image contains exactly one face.

    Args:
        image_path (str): Path to the image file

    Returns:
        bool: True if image is valid, False otherwise
    """
    # Check if file exists and has content
    if not os.path.exists(image_path):
        return False

    if os.path.getsize(image_path) == 0:
        return False

    # Try to load image
    img = cv2.imread(image_path)
    if img is None:
        return False

    # Convert to RGB for face_recognition
    rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Detect faces
    face_locations = face_recognition.face_locations(rgb)
    face_count = len(face_locations)

    # Should have exactly one face
    return face_count == 1

def encode_single_face(image_path):
    """
    Encode a single face from an image.

    Args:
        image_path (str): Path to the image file

    Returns:
        numpy.ndarray: Face encoding or None if no face found
    """
    img = cv2.imread(image_path)
```

```python
    if img is None:
        return None

    # Convert to RGB
    rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Try face_recognition library first
    encodings = face_recognition.face_encodings(rgb)
    if encodings:
        return encodings[0]

    # Fallback to Haar Cascades if face_recognition fails
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    if len(faces) > 0:
        x, y, w, h = faces[0]
        face_img = rgb[y:y+h, x:x+w]
        encodings = face_recognition.face_encodings(face_img)
        if encodings:
            return encodings[0]

    return None

def organize_face_data(source_folder="faces", organized_folder="faces_organized"):
    """
    Organize face data into a structured folder hierarchy.

    Args:
        source_folder (str): Source folder containing face images
        organized_folder (str): Destination folder for organized data
    """
    print(f"[INFO] Organizing face data from {source_folder} to {organized_folder}")

    # Create organized folder structure
    os.makedirs(organized_folder, exist_ok=True)

    # Process each user folder
    for user_folder in os.listdir(source_folder):
        user_path = os.path.join(source_folder, user_folder)

        if not os.path.isdir(user_path):
            continue
```

```python

        print(f"\n[INFO] Organizing user: {user_folder}")

        # Create user folder in organized structure
        organized_user_path = os.path.join(organized_folder, user_folder)
        os.makedirs(organized_user_path, exist_ok=True)

        # Copy and organize images
        image_count = 0
        for img_file in os.listdir(user_path):
            if img_file.lower().endswith('.jpg', '.jpeg', '.png'):
                source_img_path = os.path.join(user_path, img_file)
                dest_img_path = os.path.join(organized_user_path, f"{user_folder}_{image_count + 1}.jpg")

                # Copy image
                img = cv2.imread(source_img_path)
                if img is not None:
                    cv2.imwrite(dest_img_path, img)
                    image_count += 1
                    print(f"  Copied: {img_file} -> {user_folder}_{image_count}.jpg")

        print(f"  [OK] Organized {image_count} images for {user_folder}")

def test_multiple_expressions():
    """
    Test encoding with different facial expressions.
    """
    print("[INFO] Testing encoding with different expressions...")

    # Test with existing data
    test_folder = "faces"
    if not os.path.exists(test_folder):
        print(f"[ERROR] Test folder {test_folder} does not exist")
        return

    # Encode all images
    results = encode_folder_to_pkl(test_folder, "data/test_encodings.pkl")

    # Analyze results by user
```

```
    print(f"\n=== EXPRESSION TEST RESULTS ===")
    for user in results['users_encoded']:
        print(f"\nUser: {user}")
        # Load encodings to analyze
        try:
            with open("data/test_encodings.pkl", 'rb') as f:
                encodings = pickle.load(f)

            if user in encodings:
                user_encodings = encodings[user]
                print(f"  Total encodings: {len(user_encodings)}")

                # Show encoding details
                for i, encoding_data in enumerate(user_encodings):
                    img_path = encoding_data['image_path']
                    img_name = os.path.basename(img_path)
                    print(f"    {i+1}. {img_name}")

        except Exception as e:
            print(f"  Error loading encodings: {e}")

def main():
    """
    Main function for batch encoding operations.
    """
    print("=== Batch Face Encoding Utility ===")
    print("1. Encode all images in faces/ folder")
    print("2. Organize face data")
    print("3. Test multiple expressions")
    print("4. Custom folder encoding")

    choice = input("\nSelect operation (1-4): ").strip()

    if choice == "1":
        # Encode faces folder
        results = encode_folder_to_pkl("faces", "data/encodings.pkl")

    elif choice == "2":
        # Organize face data
        organize_face_data("faces", "faces_organized")

    elif choice == "3":
        # Test multiple expressions
        test_multiple_expressions()

```

```
306.      elif choice == "4":
307.          # Custom folder encoding
308.          folder = input("Enter folder path: ").strip()
309.          output = input("Enter output .pkl path (default:
   data/custom_encodings.pkl): ").strip()
310.          if not output:
311.              output = "data/custom_encodings.pkl"
312.
313.          if os.path.exists(folder):
314.              results = encode_folder_to_pkl(folder, output)
315.          else:
316.              print(f"[ERROR] Folder {folder} does not exist")
317.
318.      else:
319.          print("Invalid choice")
320.
321.  if __name__ == "__main__":
322.      main()
323.
```

2. Encoding_test

```python
#!/usr/bin/env python3
"""
Face Encoding Test with Multiple Expressions
Tests encoding with different facial expressions and organizes data.
"""

import os
import pickle
from batch_encoder import encode_folder_to_pkl, organize_face_data
from face_manager import recognize_face, list_users

def test_encoding_accuracy():
    """
    Test encoding accuracy with different expressions.
    """
    print("=== Face Encoding Accuracy Test ===")

    # Test with organized data
    test_folder = "faces_organized"
    if not os.path.exists(test_folder):
        print(f"[ERROR] Test folder {test_folder} does not exist")
        return

    # Encode all images
    print("\n1. Encoding all images...")
    results = encode_folder_to_pkl(test_folder, "data/test_encodings.pkl")

    if results['successful_encodings'] == 0:
        print("[ERROR] No successful encodings found")
        return

    # Test recognition accuracy
    print("\n2. Testing recognition accuracy...")
    test_recognition_accuracy("data/test_encodings.pkl")

    # Analyze encoding consistency
    print("\n3. Analyzing encoding consistency...")
    analyze_encoding_consistency("data/test_encodings.pkl")

def test_recognition_accuracy(encodings_file):
    """
    Test recognition accuracy using the generated encodings.
    """
    try:
        with open(encodings_file, 'rb') as f:
```

```
            encodings_data = pickle.load(f)

        print(f"Loaded encodings for {len(encodings_data)} users")

        # Test each user's images
        for user, user_encodings in encodings_data.items():
            print(f"\nTesting user: {user}")
            print(f"  Total encodings: {len(user_encodings)}")

            # Test recognition on each image
            correct_recognitions = 0
            for i, encoding_data in enumerate(user_encodings):
                img_path = encoding_data['image_path']
                img_name = os.path.basename(img_path)

                # Test recognition
                result = recognize_face(img_path)

                if result == user:
                    correct_recognitions += 1
                    status = "[OK]"
                else:
                    status = "[ERROR]"

                print(f"    {i+1}. {img_name}: {result} {status}")

            accuracy = (correct_recognitions / len(user_encodings)) * 100
            print(f"  Recognition accuracy: {accuracy:.1f}% ({correct_recognitions}/{len(user_encodings)})")

    except Exception as e:
        print(f"[ERROR] Failed to test recognition: {e}")

def analyze_encoding_consistency(encodings_file):
    """
    Analyze consistency between different encodings of the same user.
    """
    try:
        with open(encodings_file, 'rb') as f:
            encodings_data = pickle.load(f)

        import numpy as np

        for user, user_encodings in encodings_data.items():
            if len(user_encodings) < 2:
                continue

```

```python
            print(f"\nAnalyzing consistency for user: {user}")

            # Calculate distances between encodings
            encodings = [data['encoding'] for data in user_encodings]
            distances = []

                for i in range(len(encodings)):
                    for j in range(i + 1, len(encodings)):
                        distance = np.linalg.norm(encodings[i] - encodings[j])
                        distances.append(distance)

                if distances:
                    avg_distance = np.mean(distances)
                    min_distance = np.min(distances)
                    max_distance = np.max(distances)

                    print(f"  Average distance: {avg_distance:.4f}")
                    print(f"  Min distance: {min_distance:.4f}")
                    print(f"  Max distance: {max_distance:.4f}")

                    # Consistency rating
                    if avg_distance < 0.4:
                        consistency = "High"
                    elif avg_distance < 0.6:
                        consistency = "Medium"
                    else:
                        consistency = "Low"

                    print(f"  Consistency rating: {consistency}")

        except Exception as e:
            print(f"[ERROR] Failed to analyze consistency: {e}")

    def create_expression_test_data():
        """
        Create test data structure for different expressions.
        """
        print("=== Creating Expression Test Data Structure ===")

        # Create organized folder structure
        test_structure = {
            "faces_expressions": {
                "user1_neutral": ["neutral_1.jpg", "neutral_2.jpg"],
                "user1_smiling": ["smile_1.jpg", "smile_2.jpg"],
                "user2_neutral": ["neutral_1.jpg", "neutral_2.jpg"],
                "user2_smiling": ["smile_1.jpg", "smile_2.jpg"]
            }
```

```
        }

        print("Recommended folder structure for expression testing:")
        print("faces_expressions/")
        print("├── user1_neutral/")
        print("│   ├── neutral_1.jpg")
        print("│   └── neutral_2.jpg")
        print("├── user1_smiling/")
        print("│   ├── smile_1.jpg")
        print("│   └── smile_2.jpg")
        print("├── user2_neutral/")
        print("│   ├── neutral_1.jpg")
        print("│   └── neutral_2.jpg")
        print("└── user2_smiling/")
        print("    ├── smile_1.jpg")
        print("    └── smile_2.jpg")

        print("\nThis structure allows testing:")
        print("- Same user with different expressions")
        print("- Cross-expression recognition accuracy")
        print("- Expression-specific encoding consistency")

    def demonstrate_batch_encoding():
        """
        Demonstrate the batch encoding functionality.
        """
        print("=== Batch Encoding Demonstration ===")

        # Show current data organization
        print("\n1. Current data organization:")
        if os.path.exists("faces_organized"):
            for user_folder in os.listdir("faces_organized"):
                user_path = os.path.join("faces_organized", user_folder)
                if os.path.isdir(user_path):
                    image_count = len([f for f in os.listdir(user_path) if f.lower().endswith(('.jpg', '.jpeg', '.png'))])
                    print(f"  {user_folder}: {image_count} images")

        # Show encoding results
        print("\n2. Encoding results:")
        if os.path.exists("data/organized_encodings.pkl"):
            try:
                with open("data/organized_encodings.pkl", 'rb') as f:
                    encodings = pickle.load(f)

                for user, user_encodings in encodings.items():
```

```python
                    print(f"  {user}: {len(user_encodings)} encodings")
                    for i, encoding_data in enumerate(user_encodings):
                        img_name = os.path.basename(encoding_data['image_path'])
                        timestamp = encoding_data['timestamp'][:19]  # Remove microseconds
                        print(f"    {i+1}. {img_name} (encoded: {timestamp})")

            except Exception as e:
                print(f"  Error loading encodings: {e}")

        # Show recognition test
        print("\n3. Recognition test:")
        if os.path.exists("faces_organized/charan"):
            charan_images = [f for f in os.listdir("faces_organized/charan") if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
            if charan_images:
                test_image = os.path.join("faces_organized/charan", charan_images[0])
                result = recognize_face(test_image)
                print(f"  Test image: {charan_images[0]}")
                print(f"  Recognition result: {result}")

    def main():
        """
        Main function for testing batch encoding.
        """
        print("=== Face Encoding Test Suite ===")
        print("1. Test encoding accuracy")
        print("2. Create expression test data structure")
        print("3. Demonstrate batch encoding")
        print("4. Run all tests")

        choice = input("\nSelect test (1-4): ").strip()

        if choice == "1":
            test_encoding_accuracy()
        elif choice == "2":
            create_expression_test_data()
        elif choice == "3":
            demonstrate_batch_encoding()
        elif choice == "4":
            test_encoding_accuracy()
            create_expression_test_data()
```

```
            demonstrate_batch_encoding()
        else:
            print("Invalid choice")

    if __name__ == "__main__":
        # Run demonstration automatically
        demonstrate_batch_encoding()
        print("\n" + "="*50)
        test_encoding_accuracy()
```

3. Face_manager

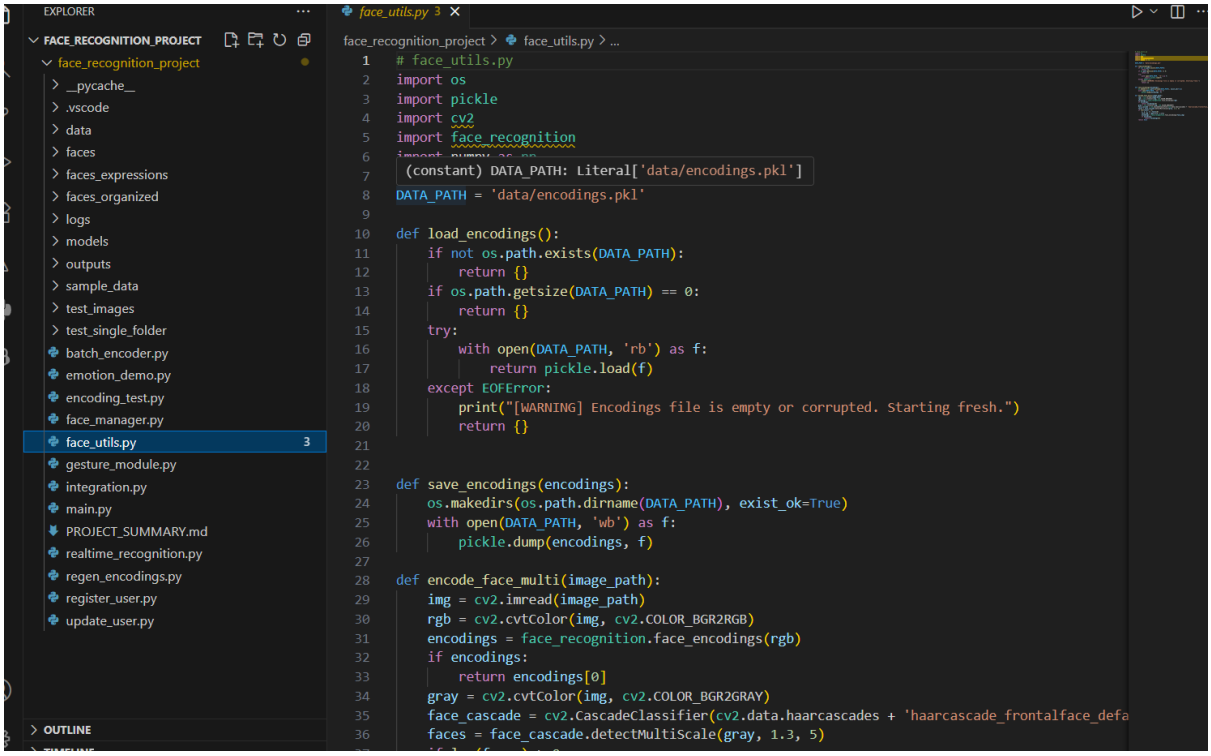
```
# face_manager.py
from face_utils import load_encodings, save_encodings, encode_face_multi
import numpy as np

def add_user(user_id, image_path):
    encodings = load_encodings()
    encoding = encode_face_multi(image_path)
    if encoding is None:
        raise ValueError("No face found in the image")
    encodings[user_id] = encoding
    save_encodings(encodings)
    print(f"[INFO] User {user_id} added.")

def remove_user(user_id):
    encodings = load_encodings()
    if user_id in encodings:
        del encodings[user_id]
        save_encodings(encodings)
        print(f"[INFO] User {user_id} removed.")

def update_user(user_id, image_path):
    encodings = load_encodings()
    if user_id in encodings:
        encoding = encode_face_multi(image_path)
        encodings[user_id] = encoding
        save_encodings(encodings)
        print(f"[INFO] User {user_id} updated.")
    else:
        print("[WARN] User not found. Use add_user to register.")

def list_users():
    encodings = load_encodings()
    return list(encodings.keys())

def recognize_face(image_path, tolerance=0.6):
    unknown_encoding = encode_face_multi(image_path)
    if unknown_encoding is None:
        return None

    encodings_dict = load_encodings()
    if not encodings_dict:
        return "Unknown"

    user_ids = list(encodings_dict.keys())
    encodings = np.array(list(encodings_dict.values()))
```

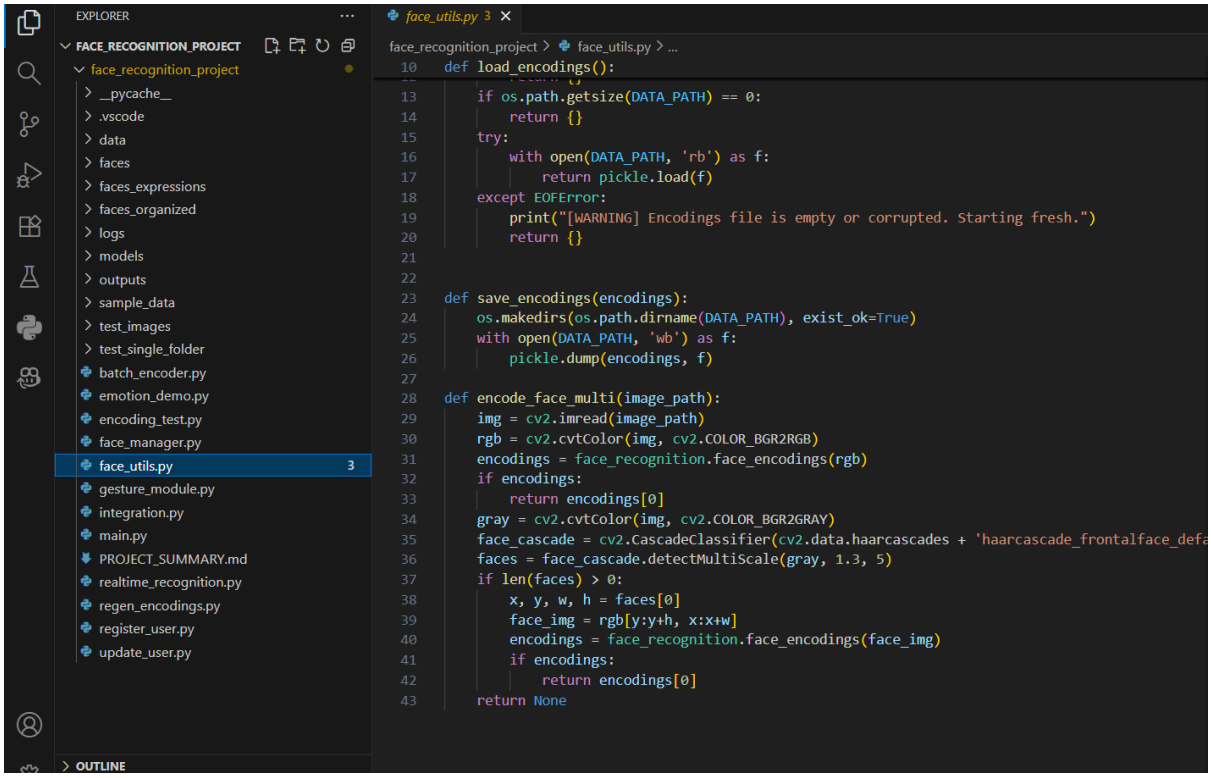
```
    distances = np.linalg.norm(encodings - unknown_encoding, axis=1)
    idx = np.where(distances <= tolerance)[0]

    if len(idx) > 0:
        return user_ids[idx[0]]
    return "Unknown"
# face_manager.py
from face_utils import load_encodings, save_encodings, encode_face_multi
import numpy as np

def add_user(user_id, image_path):
    encodings = load_encodings()
    encoding = encode_face_multi(image_path)
    if encoding is None:
        raise ValueError("No face found in the image")
    encodings[user_id] = encoding
    save_encodings(encodings)
    print(f"[INFO] User {user_id} added.")

def remove_user(user_id):
    encodings = load_encodings()
    if user_id in encodings:
        del encodings[user_id]
        save_encodings(encodings)
        print(f"[INFO] User {user_id} removed.")

def update_user(user_id, image_path):
    encodings = load_encodings()
    if user_id in encodings:
        encoding = encode_face_multi(image_path)
        encodings[user_id] = encoding
        save_encodings(encodings)
        print(f"[INFO] User {user_id} updated.")
    else:
        print("[WARN] User not found. Use add_user to register.")

def list_users():
    encodings = load_encodings()
    return list(encodings.keys())

def recognize_face(image_path, tolerance=0.6):
    unknown_encoding = encode_face_multi(image_path)
    if unknown_encoding is None:
        return None

    encodings_dict = load_encodings()
    if not encodings_dict:
```

```
        return "Unknown"

    user_ids = list(encodings_dict.keys())
    encodings = np.array(list(encodings_dict.values()))
        distances = np.linalg.norm(encodings - unknown_encoding, axis=1)
        idx = np.where(distances <= tolerance)[0]

        if len(idx) > 0:
            return user_ids[idx[0]]
        return "Unknown"
    # face_manager.py
    from face_utils import load_encodings, save_encodings, encode_face_multi
    import numpy as np

    def add_user(user_id, image_path):
        encodings = load_encodings()
        encoding = encode_face_multi(image_path)
        if encoding is None:
            raise ValueError("No face found in the image")
        encodings[user_id] = encoding
        save_encodings(encodings)
        print(f"[INFO] User {user_id} added.")

    def remove_user(user_id):
        encodings = load_encodings()
        if user_id in encodings:
            del encodings[user_id]
            save_encodings(encodings)
            print(f"[INFO] User {user_id} removed.")

    def update_user(user_id, image_path):
        encodings = load_encodings()
        if user_id in encodings:
            encoding = encode_face_multi(image_path)
            encodings[user_id] = encoding
            save_encodings(encodings)
            print(f"[INFO] User {user_id} updated.")
        else:
            print("[WARN] User not found. Use add_user to register.")

    def list_users():
        encodings = load_encodings()
        return list(encodings.keys())

    def recognize_face(image_path, tolerance=0.6):
        unknown_encoding = encode_face_multi(image_path)
```

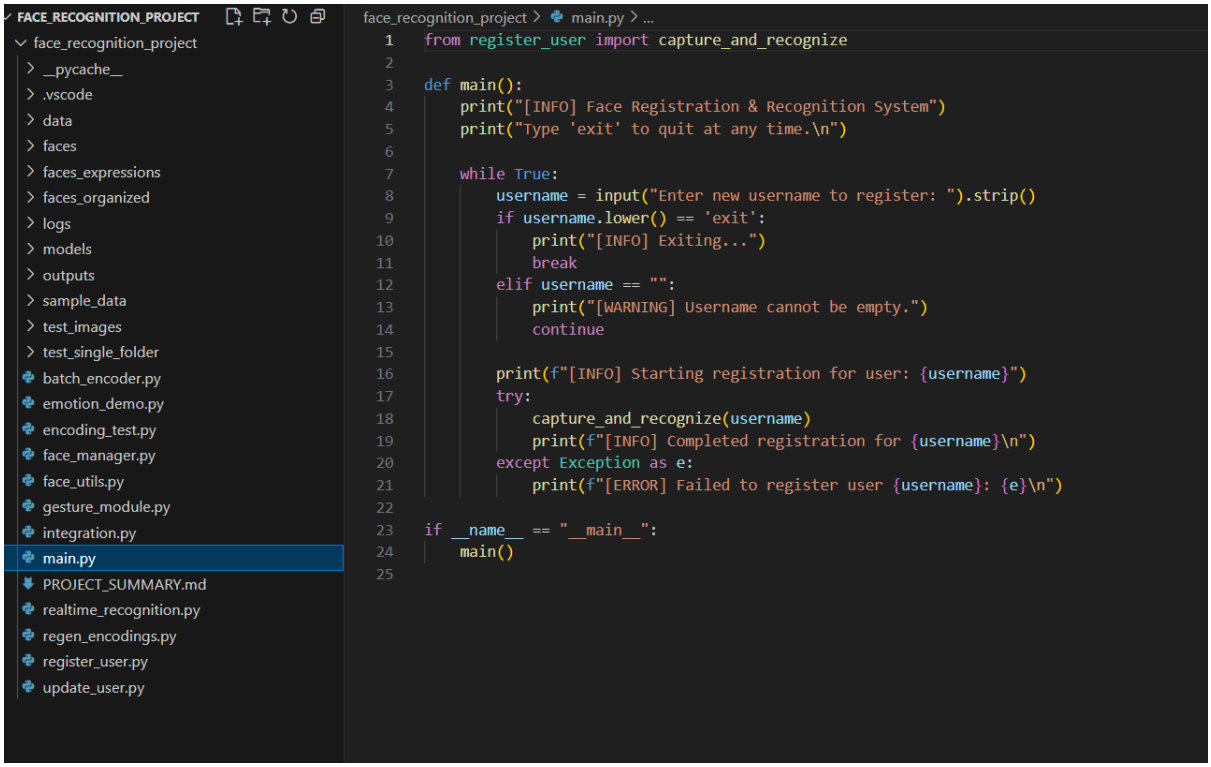
```
        if unknown_encoding is None:
            return None

        encodings_dict = load_encodings()
        if not encodings_dict:
            return "Unknown"

        user_ids = list(encodings_dict.keys())
        encodings = np.array(list(encodings_dict.values()))
        distances = np.linalg.norm(encodings - unknown_encoding, axis=1)
        idx = np.where(distances <= tolerance)[0]

        if len(idx) > 0:
            return user_ids[idx[0]]
        return "Unknown"
```

4. face_utils

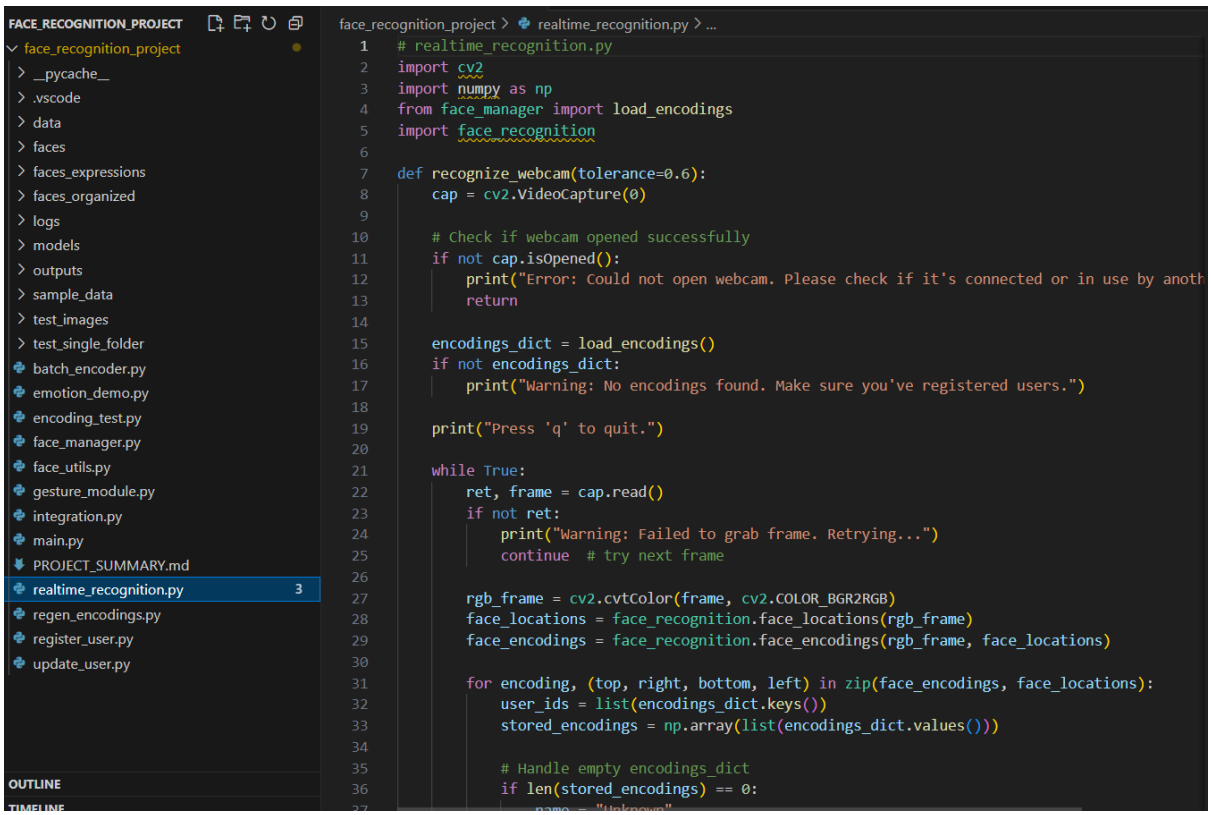

EXPLORER
FACE_RECOGNITION_PROJECT
face_recognition_project
__pycache__
.vscode
data
faces
faces_expressions
faces_organized
logs
models
outputs
sample_data
test_images
test_single_folder
batch_encoder.py
emotion_demo.py
encoding_test.py
face_manager.py
face_utils.py 3
gesture_module.py
integration.py
main.py
PROJECT_SUMMARY.md
realtime_recognition.py
regen_encodings.py
register_user.py
update_user.py
OUTLINE
face_utils.py 3
face_recognition_project > face_utils.py > ...
# face_utils.py
import os
import pickle
import cv2
import face_recognition
import numpy as np
(constant) DATA_PATH: Literal['data/encodings.pkl']
DATA_PATH = 'data/encodings.pkl'
def load_encodings():
if not os.path.exists(DATA_PATH):
return {}
if os.path.getsize(DATA_PATH) == 0:
return {}
try:
with open(DATA_PATH, 'rb') as f:
return pickle.load(f)
except EOFError:
print("[WARNING] Encodings file is empty or corrupted. Starting fresh.")
return {}
def save_encodings(encodings):
os.makedirs(os.path.dirname(DATA_PATH), exist_ok=True)
with open(DATA_PATH, 'wb') as f:
pickle.dump(encodings, f)
def encode_face_multi(image_path):
img = cv2.imread(image_path)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
encodings = face_recognition.face_encodings(rgb)
if encodings:
return encodings[0]
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_defa
faces = face_cascade.detectMultiScale(gray, 1.3, 5)


EXPLORER
FACE_RECOGNITION_PROJECT
face_recognition_project
__pycache__
.vscode
data
faces
faces_expressions
faces_organized
logs
models
outputs
sample_data
test_images
test_single_folder
batch_encoder.py
emotion_demo.py
encoding_test.py
face_manager.py
face_utils.py 3
gesture_module.py
integration.py
main.py
PROJECT_SUMMARY.md
realtime_recognition.py
regen_encodings.py
register_user.py
update_user.py
OUTLINE
face_utils.py 3
face_recognition_project > face_utils.py > ...
def load_encodings():
if os.path.getsize(DATA_PATH) == 0:
return {}
try:
with open(DATA_PATH, 'rb') as f:
return pickle.load(f)
except EOFError:
print("[WARNING] Encodings file is empty or corrupted. Starting fresh.")
return {}
def save_encodings(encodings):
os.makedirs(os.path.dirname(DATA_PATH), exist_ok=True)
with open(DATA_PATH, 'wb') as f:
pickle.dump(encodings, f)
def encode_face_multi(image_path):
img = cv2.imread(image_path)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
encodings = face_recognition.face_encodings(rgb)
if encodings:
return encodings[0]
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_defa
faces = face_cascade.detectMultiScale(gray, 1.3, 5)
if len(faces) > 0:
x, y, w, h = faces[0]
face_img = rgb[y:y+h, x:x+w]
encodings = face_recognition.face_encodings(face_img)
if encodings:
return encodings[0]
return None

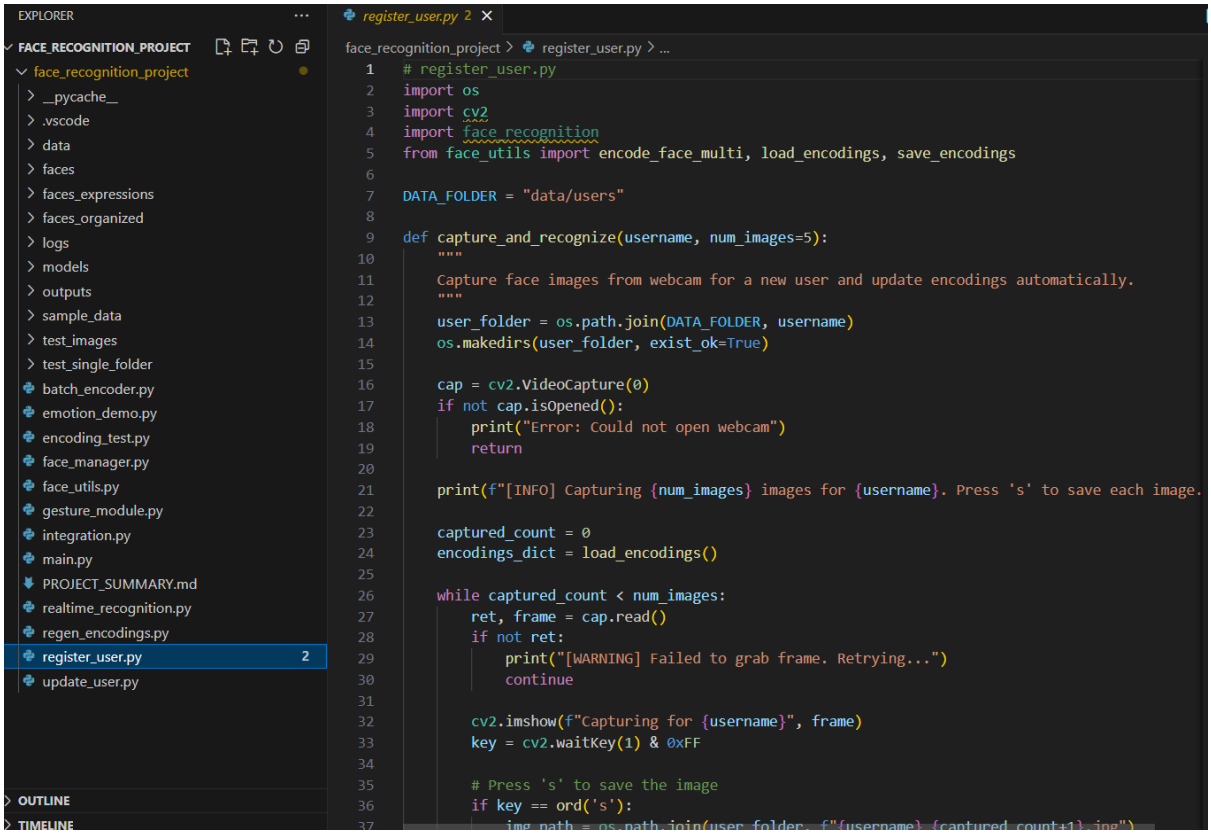
5. main.py


FACE_RECOGNITION_PROJECT
face_recognition_project
__pycache__
.vscode
data
faces
faces_expressions
faces_organized
logs
models
outputs
sample_data
test_images
test_single_folder
batch_encoder.py
emotion_demo.py
encoding_test.py
face_manager.py
face_utils.py
gesture_module.py
integration.py
main.py
PROJECT_SUMMARY.md
realtime_recognition.py
regen_encodings.py
register_user.py
update_user.py
face_recognition_project > main.py > ...
from register_user import capture_and_recognize
def main():
print("[INFO] Face Registration & Recognition System")
print("Type 'exit' to quit at any time.\n")
while True:
username = input("Enter new username to register: ").strip()
if username.lower() == 'exit':
print("[INFO] Exiting...")
break
elif username == "":
print("[WARNING] Username cannot be empty.")
continue
print(f"[INFO] Starting registration for user: {username}")
try:
capture_and_recognize(username)
print(f"[INFO] Completed registration for {username}\n")
except Exception as e:
print(f"[ERROR] Failed to register user {username}: {e}\n")
if __name__ == "__main__":
main()

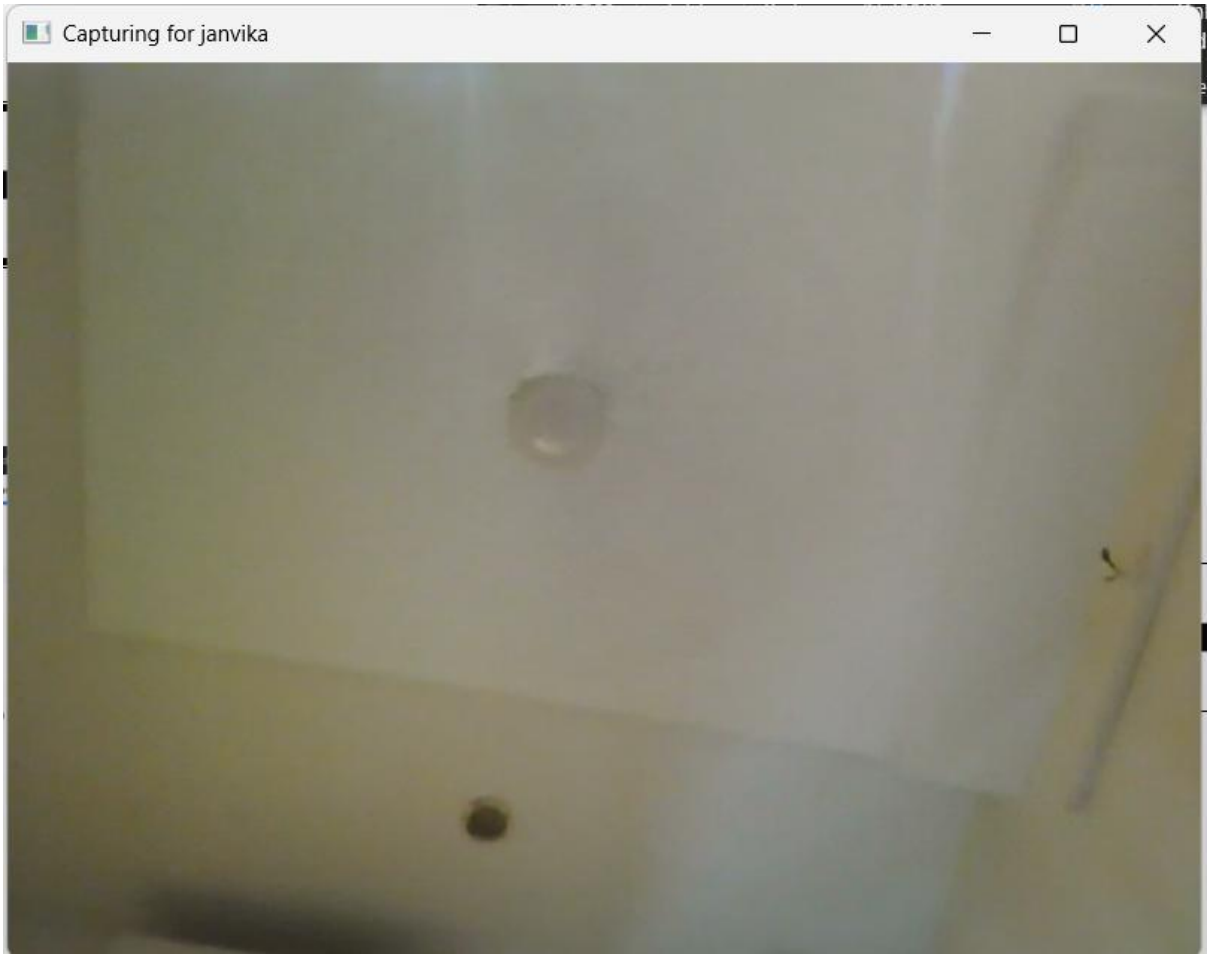
6. realtime_recognition



8.register_user



9.Outputs



CHAPTER 6

CONCLUSION

Conclusion:

The Face Recognition System successfully demonstrates the power of **Artificial Intelligence** and **Computer Vision** in automating identity verification.
By using facial features for authentication, the project provides a **secure, contactless, and efficient alternative** to traditional methods like passwords or ID cards.

Through real-time detection, encoding, and recognition, the system ensures **high accuracy and faster processing**, making it applicable in areas such as **attendance systems, surveillance, and access control**.
Although it has certain limitations related to lighting and hardware, the project lays a strong foundation for **future enhancements** like emotion detection, cloud integration, and multi-user scalability.

In conclusion, this project showcases how AI-based face recognition can **enhance security, reduce manual effort, and promote smarter automation** in real-world applications.